# Firebase Installations (Firebase Instance Id) Implementation (Android & Flutter[App])

Created
**Version 1**: 06 Apr 2026

Table of Contents

**Overview** **3**

**Implementation steps** **4**

Add Firebase Installation in a Android App 4
Register the App for Java Android with Firebase & Configuration File 5
Configuration of code with snippet in Java Android 6
Firebase Installation Output in a Java Android 7
Register the App for Flutter with Firebase & Configuration File 8
Configuration of code with snippet in Flutter 10
Firebase Installation Output in Flutter 14

# Overview

- Firebase Instance ID provides a unique identifier for each app instance and a mechanism to authenticate and authorize actions.
- Firebase Instance Id is deprecated and furthermore now it's renowned as a Firebase Installation.
- Firebase Installation is a modern replacement for Instance ID, providing a **Firebase Installation ID (FID)** for every app instance.
- Firebase Installation is improved in better reliability, security, and performance compared to the older system.
- Firebase Installations primarily used in several firebase services in **Analytics, Crashlytics, Cloud Messaging, Remote Config** also.
- Firebase Installation is their functionality beyond providing IDs, it can generate **Installation Auth Tokens** to authenticate specific app instances without requiring user login.

**Note:** The document contains sample static values, at the time of implementation use the dynamic values.

# Add Firebase Installation in a Android App

## Firebase Installation in android

Before the integration of firebase installation your app must be integrated with firebase along with their firebase analytics attached with their dependencies.
If you did not integrate your app with firebase and their feature which has been given above, I will mention steps for integration of firebase.

### Step-by-Step Integration

Step 1: Project Setup
1.Create a Firebase Project
Go to Firebase Console
Click "Add project" or select existing project
Follow the setup wizard
2. Register Your Android App
In Firebase Console, click "Add app" → Android icon
Enter your app's package name (found in AndroidManifest.xml)
To add SHA-1 certificate
Click "Register app"
3. Download google-services.json
Download the google-services.json file
Step 2: Configure Gradle Files
2.1 Project-level build.gradle (build.gradle in root directory)

```
gradle
dependencies {
classpath 'com.android.tools.build:gradle:8.1.0'
classpath 'com.google.gms:google-services:4.4.0'  // Add this
}
}
```

2.2 App-level build.gradle (`app/build.gradle`)

```
gradle
plugins {
id 'com.android.application'
id 'com.google.gms.google-services' // Add this at the bottom
}
dependencies {
```

```
// Firebase BoM (Bill of Materials) - manages versions
implementation platform('com.google.firebase:firebase-bom:32.7.0')
// Firebase Installations
implementation 'com.google.firebase:firebase-installations'
// Firebase Analytics (optional but recommended)
implementation 'com.google.firebase:firebase-analytics'
// Other dependencies
implementation 'androidx.appcompat:appcompat:1.6.1'
implementation 'com.google.android.material:material:1.11.0'
}
```

## 1.2 Configuration of code with snippet in Java Android

### Scenario 1

AndroidMainfest.xml

```
XML
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

### Scenario 2

mainActivity.java

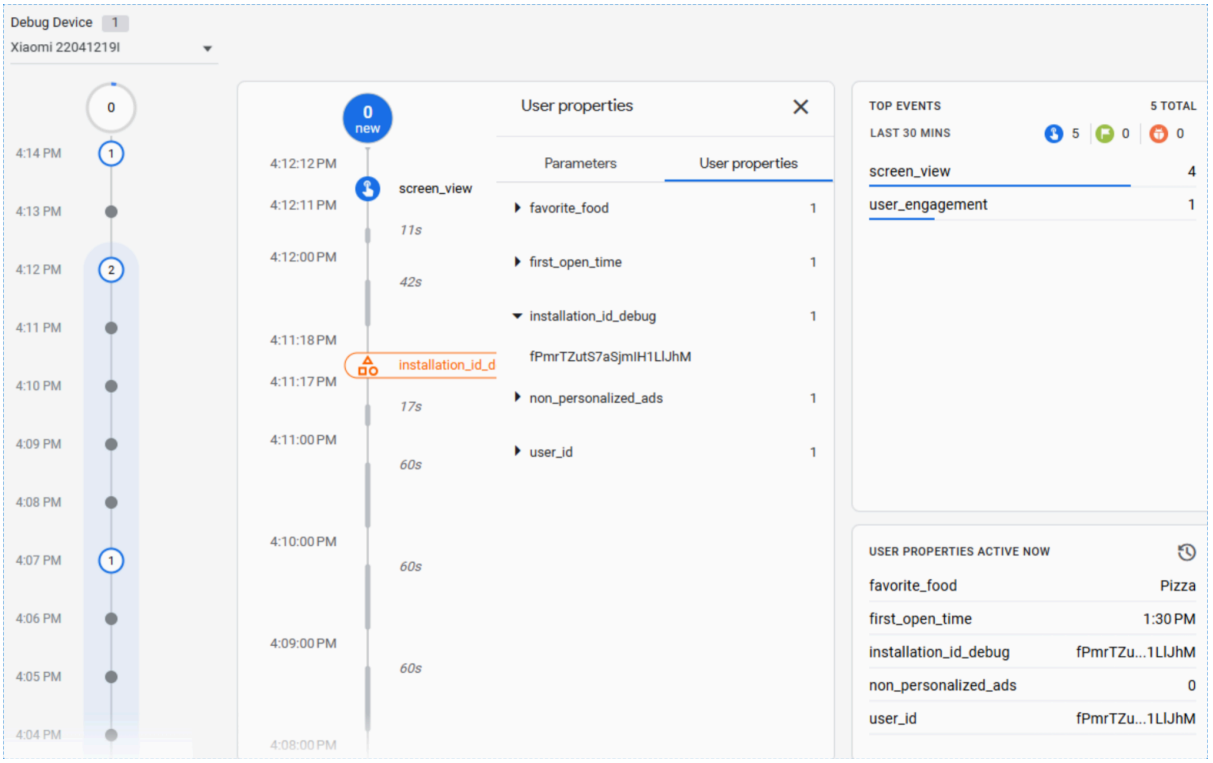
```
Java
public class MyApplication extends Application {
import com.google.firebase.installations.FirebaseInstallations;
private String firebaseInstallationId;
@Override
private void fetchFirebaseId() {
FirebaseInstallations.getInstance().getId().addOnCompleteListener(task -> {
if (task.isSuccessful()) {
firebaseInstallationId = task.getResult();
Log.d("FirebaseReady", "Installation ID: " + firebaseInstallationId);
} else {
Log.e("FirebaseReady",
"Unable to get Installation ID", task.getException());
}
});
}
public FirebaseAnalytics getAnalytics() {
return mFirebaseAnalytics;
}
}
private void fetchFirebaseInstallationId() {
FirebaseInstallations.getInstance().getId()
```

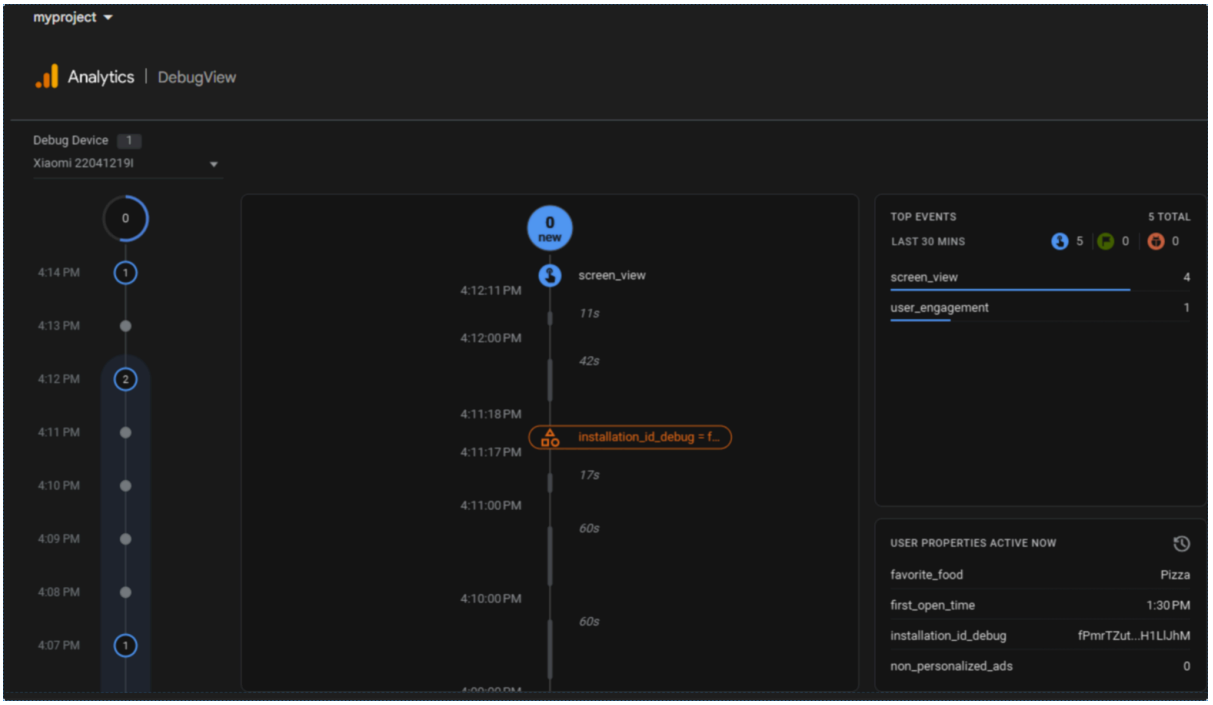
```
.addOnCompleteListener(new OnCompleteListener<String>() {
@Override
 public void onComplete(@NonNull Task<String> task) {
if (task.isSuccessful()) {
String fid = task.getResult();
Log.d(TAG, "Installation ID: " + fid);
// Send to Firebase DebugView
mFirebaseAnalytics.setUserProperty("installation_id_debug", fid);
mFirebaseAnalytics.setUserId(fid);
Toast.makeText(MainActivity.this, "ID: " + fid, Toast.LENGTH_LONG).show();
} else {
Log.e(TAG, "Fetch failed", task.getException());
Toast.makeText(MainActivity.this,
"Fetch Failed", Toast.LENGTH_SHORT).show();
}
}
});
}
```

## 1.3 Firebase Installation Output in a Java Android

Firebase Installation Id implementation snippets

```
40          private void fetchFirebaseInstallationId() {
41              FirebaseInstallations.getInstance().getId()
42                      .addOnCompleteListener(new OnCompleteListener<String>() {
43                          @Override
44                          public void onComplete(@NonNull Task<String> task) {
45                              if (task.isSuccessful()) {
46                                  String fid = task.getResult();
47                                  Log.d(TAG, msg: "Installation ID: " + fid);
48                                  // Send to Firebase DebugView
49                                  mFirebaseAnalytics.setUserProperty( name: "installation_id_debug", fid);
50                                  mFirebaseAnalytics.setUserId(fid);
51                                  Toast.makeText( context: MainActivity.this,  text: "ID: " + fid, Toast.LENGTH_LONG).show();
52                              } else {
53                                  Log.e(TAG, msg: "Fetch failed", task.getException());
54                                  Toast.makeText( context: MainActivity.this,  text: "Fetch Failed", Toast.LENGTH_SHORT).show();
55                              }
56                          }
57                      });
58          }
59      }
```


Debug Device 1
Xiaomi 22041219I
0
4:14 PM
4:13 PM
4:12 PM
4:11 PM
4:10 PM
4:09 PM
4:08 PM
4:07 PM
4:06 PM
4:05 PM
4:04 PM
0 new
4:12:12 PM
screen_view
4:12:11 PM
11s
4:12:00 PM
42s
4:11:18 PM
installation_id_d
4:11:17 PM
17s
4:11:00 PM
60s
4:10:00 PM
60s
4:09:00 PM
60s
4:08:00 PM
User properties
Parameters
User properties
favorite_food 1
first_open_time 1
installation_id_debug 1
fPmrTZutS7aSjmIH1LlJhM
non_personalized_ads 1
user_id 1
TOP EVENTS
5 TOTAL
LAST 30 MINS
5 0 0
screen_view 4
user_engagement 1
USER PROPERTIES ACTIVE NOW
favorite_food Pizza
first_open_time 1:30 PM
installation_id_debug fPmrTZu...1LlJhM
non_personalized_ads 0
user_id fPmrTZu...1LlJhM


myproject
Analytics | DebugView
Debug Device 1
Xiaomi 22041219I
0
4:14 PM
4:13 PM
4:12 PM
4:11 PM
4:10 PM
4:09 PM
4:08 PM
4:07 PM
0 new
screen_view
4:12:11 PM
11s
4:12:00 PM
42s
4:11:18 PM
installation_id_debug = f...
4:11:17 PM
17s
4:11:00 PM
60s
4:10:00 PM
60s
TOP EVENTS
5 TOTAL
LAST 30 MINS
5 0 0
screen_view 4
user_engagement 1
USER PROPERTIES ACTIVE NOW
favorite_food Pizza
first_open_time 1:30 PM
installation_id_debug fPmrTZut...H1LlJhM
non_personalized_ads 0

## 2 Firebase Installation in flutter

The Firebase Installation is also working upon Cross - Platform such as Flutter.

All Steps which i did in Android Java as it is same in Flutter which i mentioned below regarding steps, codes and dependencies.

Step-by-Step Integration:

1. Create a Firebase Project

Go to Firebase Console

Click "Add project" or select existing project

### 2. Register Your Android App

In Firebase Console, click "Add app" → Android icon

Enter your app's package name (found in AndroidManifest.xml)

To add SHA-1 certificate

Click "Register app"

### 3. Download google-services.json

Add Firebase to Flutter using CLI (Recommended)

Install FlutterFire CLI:

```
dart pub global activate flutterfire_cli
Then run:
```

```
flutterfire configure
```

This will:

Automatically link your Flutter app with Firebase

Generate firebase_options.dart

Handle most Gradle setup for you

### 2.2 Add Required Dependencies

In your pubspec.yaml:

```
dependencies:
 flutter:
 sdk: flutter
 firebase_core: ^latest_version
Then run:
flutter pub get
```

### Step 3: Initialize Firebase in Flutter

```
main.dart
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';
void main() async {
WidgetsFlutterBinding.ensureInitialized();
await Firebase.initializeApp(
options: DefaultFirebaseOptions.currentPlatform,
);
runApp(MyApp());
}
```

### Step 4: (Important) Android Gradle Changes (Minimal in Flutter)

Flutter already manages most configs, but verify:

android/build.gradle (Project-level)

```
buildscript {
dependencies {
classpath 'com.google.gms:google-services:4.3.15'
}
}
```

```
android/app/build.gradle

At bottom:

apply plugin: 'com.google.gms.google-services'
```

Step 5: Run Your App

```
flutter run

If everything is correct → Firebase connects successfully
```

Step: 6 These are the dependencies which are attached to code with an integration with a code which can give appropriate output

```
shared_preferences: ^2.5.4
firebase_core: ^4.3.0
firebase_analytics: ^12.1.0
firebase_app_installations: ^0.4.1
```

## 2.2 Firebase integrate with and Firebase Installation Code

### Scenario 1

IntegrateFirebase Installation Code

```
main.dart
import 'package:firebase_analytics/firebase_analytics.dart';
import 'package:springseason/main.dart';
// Import your service and options
import 'firebase_options.dart';
import 'package:springseason/services/FirebaseInstallationsService.dart';
import 'package:springseason/viewmodels/InstallationViewModel.dart';
// 4. FIX: Initialize the Real Firebase Installations Service
final installationsService = FirebaseInstallationsService();
await installationsService.init();
debugPrint('Firebase Ready -> ID: ${installationsService.id}');
// 6. Start the App with Providers
runApp(
MultiProvider(
providers: [
// Provide the InstallationViewModel so screens can see the ID/Token
ChangeNotifierProvider(
create: (_) => InstallationViewModel(),
),
```

```
ChangeNotifierProvider(
create: (_) => NavigationViewModel()
),
```

## Scenario 2

In the MVVM Model Architecture in the service model there is a file firebase installation with their plugins

```
import 'dart:developer';
import 'package:firebase_app_installations/firebase_app_installations.dart';
import 'package:shared_preferences/shared_preferences.dart';
class FirebaseInstallationsService {
static final FirebaseInstallationsService _instance =
FirebaseInstallationsService._internal();
factory FirebaseInstallationsService() => _instance;
FirebaseInstallationsService._internal();
//Start with empty strings instead of 'None' to check status easily
String id = '';
// Helper to access Firebase instance
final FirebaseInstallations _installations = FirebaseInstallations.instance;
//FIX: Changed init to Future<void> and added 'await'
Future<void> init() async {
// Load existing data from storage first
await _loadFromStorage();
// Fetch fresh data from Firebase
await getId();
await getAuthToken();
// Set up the listener for background changes
_listenToChanges();
}
//PERSISTENCE: Save/Load methods
Future<void> _loadFromStorage() async {
final prefs = await SharedPreferences.getInstance();
id = prefs.getString('fb_id') ?? '';
log("Storage Loaded -> ID: $id");
}
Future<void> _saveToStorage() async {
final prefs = await SharedPreferences.getInstance();
await prefs.setString('fb_id', id);
}
//LISTENER: Moved to a separate
private method   void _listenToChanges() {
_installations.onIdChange.listen((event) async {
if (event.isNotEmpty) {
log("Firebase ID Changed: $event");
```

```
id = event;
//Force refresh token if ID changes
await _saveToStorage();
}
}).onError((error) {
log("Firebase Installation ID Error: $error");
});
}
//REFRESH LOGIC: Fixed to be safe and persistent
Future<void> getId() async {
try {
final newId = await _installations.getId();
if (newId.isNotEmpty) {
id = newId;
await _saveToStorage();
log('Installation ID: $id');
}
} catch (e) {
log('Get Installation ID Error: $e');
}
}
//DELETE: Wipes everything
Future<void> deleteId() async {
try {
await _installations.delete();
id = '';
final prefs = await SharedPreferences.getInstance();
await prefs.clear();
//Clear local storage
log('Installation ID Deleted');
} catch (e) {
log('Delete Installation ID Error: $e');
}
}
Future<void> resetAndRetrieveNewIdentity() async {}
}
```

## Scenario 3

View Model of Firebase Installation Id

```
import 'package:firebase_app_installations/firebase_app_installations.dart';
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';
class InstallationViewModel extends ChangeNotifier {
String? id;   S
```

```
String? authToken;
bool isLoading = false;
String? errorMessage;
final FirebaseInstallations _installations = FirebaseInstallations.instance;
Future<void> _safeGetId() async {
try {
// Small delay to ensure Firebase Services are fully booted
await Future.delayed(const Duration(milliseconds: 500));
await getId();
} catch (e) {
debugPrint("Safe init error: $e");
}
}
Future<void> getId() async {
try {
isLoading = true;
errorMessage = null;
notifyListeners();
// 1. Fetch the actual ID from the
Firebase Installation Service id = await _installations.getId();
if (id != null) {
debugPrint("Firebase Ready -> ID: $id");
// 2. SAVE to local storage
await _saveToPrefs();
// 3. INTEGRATE FOR DEBUG VIEW
await _analytics.setUserProperty(
name: 'installation_id_debug',
value: id);
await _analytics.setUserId(id: id);
debugPrint("DebugView: Identity parameters updated.");
}
} catch (e) {
errorMessage = "Failed to get ID: $e";
debugPrint(errorMessage);
} finally {
isLoading = false;
notifyListeners();
}
}
Future<void> deleteId() async {
try {
isLoading = true;
notifyListeners();
// 1. Delete from Firebase Servers
await _installations.delete();
// 2. Clear local data
```

```
id = null;
authToken = null;
final prefs = await SharedPreferences.getInstance();
await prefs.remove(_keyFid);
await prefs.remove(_keyToken);
// 3. Clear from Analytics DebugView
await _analytics.setUserId(id: null);
await _analytics.setUserProperty(
name: 'installation_id_debug',
value: null
);
debugPrint("Identity deleted and Analytics cleared.");
} catch (e) {
debugPrint("Delete error: $e");
} finally {
isLoading = false;
notifyListeners();
}
}
```

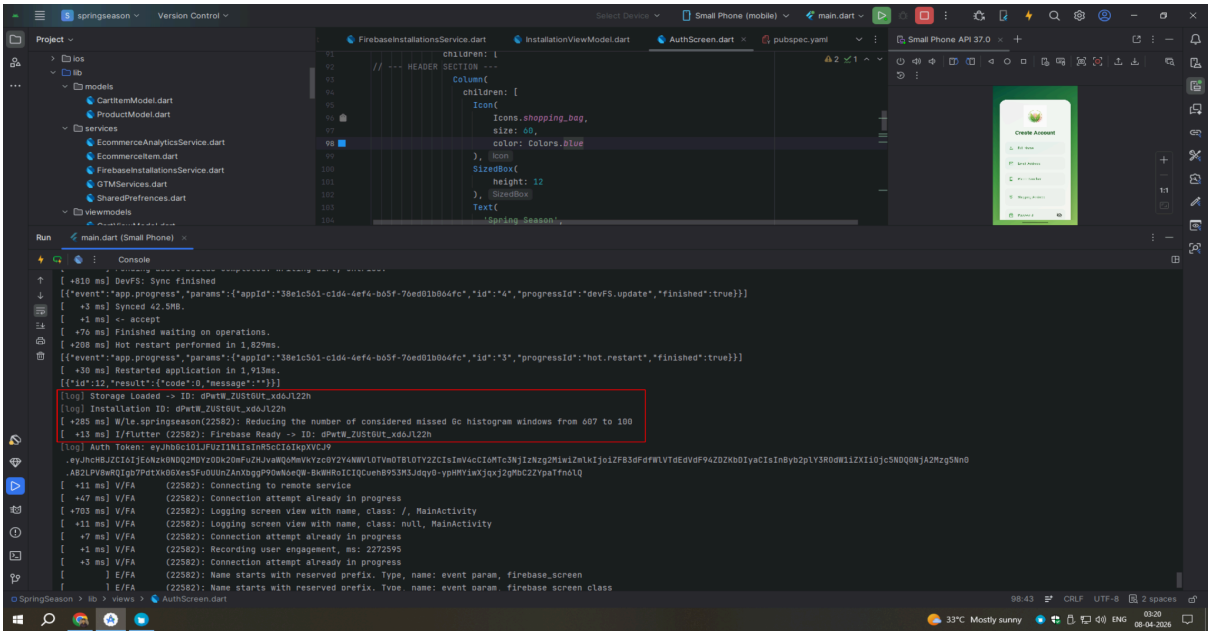
### Scenario 4

View Model of Firebase Installation Id

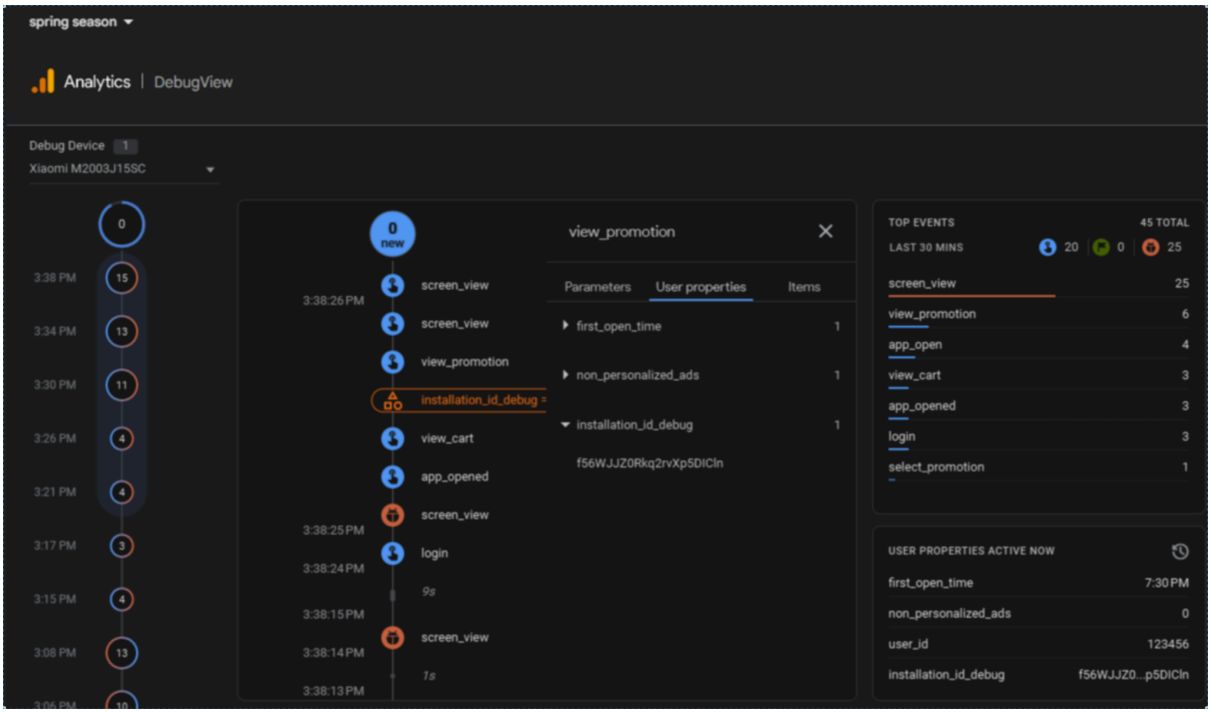
```
void _setupInitialState() async {
try {
_fid = await FirebaseInstallations.instance.getId();
if (_fid != null) {
await _analytics.setUserProperty(name: 'installation_id_debug', value: _fid);
await _analytics.setUserId(id: _fid);
debugPrint("FID Integrated: $_fid");
}
} catch (e) {
debugPrint('Installation ID Error: $e');
}
// 2. Set static User ID
try {
await _analytics.setUserId(id: '123456');
} catch (e) {
debugPrint('Analytics Error: $e');
}
```

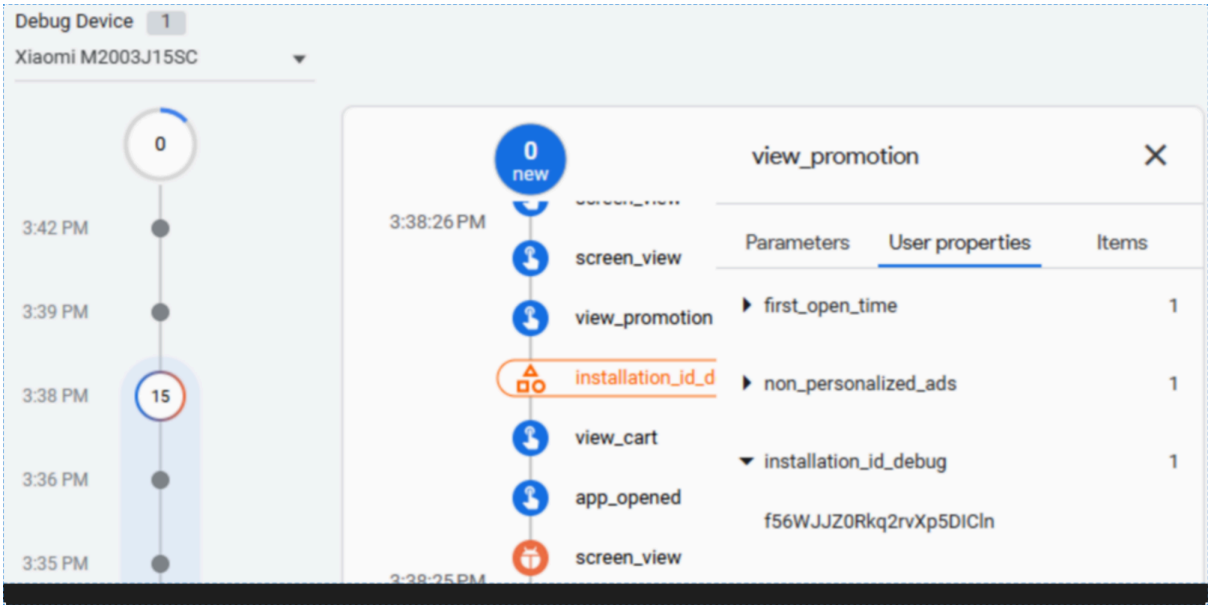
## 2.3 Firebase Installation Id Output

Implement below snippet of Firebase Installation ID


springseason
Version Control
Project
ios
lib
models
CartItemModel.dart
ProductModel.dart
services
EcommerceAnalyticsService.dart
EcommerceItem.dart
FirebaseInstallationsService.dart
GTMServices.dart
SharedPrefrences.dart
viewmodels
FirebaseInstallationsService.dart
InstallationViewModel.dart
AuthScreen.dart
pubspec.yaml
Small Phone API 37.0
// --- HEADER SECTION ---
Icon(
Icons.shopping_bag,
size: 60,
color: Colors.blue
SizedBox(
height: 12
Text(
'Spring Season',
Run
main.dart (Small Phone)
Console
[ +810 ms] DevFS: Sync finished
[ +3 ms] Synced 42.5MB.
[ +1 ms] <- accept
[ +76 ms] Finished waiting on operations.
[ +208 ms] Hot restart performed in 1,829ms.
[ +30 ms] Restarted application in 1,913ms.
[log] Storage Loaded -> ID: dPwtW_ZUSt0Ut_xd6JL22h
[log] Installation ID: dPwtW_ZUSt0Ut_xd6JL22h
[ +285 ms] W/le.springseason(22582): Reducing the number of considered missed Gc histogram windows from 607 to 100
[ +13 ms] I/flutter (22582): Firebase Ready -> ID: dPwtW_ZUSt0Ut_xd6JL22h
[log] Auth Token: eyJhbGciOiJFUzI1NiIsInRScCIoIkpXVCJ9
[ +11 ms] V/FA (22582): Connecting to remote service
[ +47 ms] V/FA (22582): Connection attempt already in progress
[ +703 ms] V/FA (22582): Logging screen view with name, class: /, MainActivity
[ +11 ms] V/FA (22582): Logging screen view with name, class: null, MainActivity
[ +7 ms] V/FA (22582): Connection attempt already in progress
[ +1 ms] V/FA (22582): Recording user engagement, ms: 2272595
[ +3 ms] V/FA (22582): Connection attempt already in progress
[ ] E/FA (22582): Name starts with reserved prefix. Type, name: event param, firebase_screen
[ ] E/FA (22582): Name starts with reserved prefix. Type, name: event param, firebase_screen_class
SpringSeason > lib > views > AuthScreen.dart
33°C Mostly sunny
ENG
03:20
08-04-2026

void _setupInitialState() async {
try {
_fid = await FirebaseInstallations.instance.getId();
if (_fid != null) {
await _analytics.setUserProperty(name: 'installation_id_debug', value: _fid);
await _analytics.setUserId(id: _fid);
debugPrint("FID Integrated: $_fid");
}
} catch (e) {
debugPrint('Installation ID Error: $e');
}
// 2. Set static User ID
try {
await _analytics.setUserId(id: '123456');
} catch (e) {
debugPrint('Analytics Error: $e');
}
// 3. GTM Call
try {
GTMService().logScreenView(screenName: 'Home', screenClass: 'HomeTab');
} catch (e) {
debugPrint("GTM Error: $e");
}
Spring Season
Product Store
Men
Women
Kids
WELCOME TO
2026
Happy New Year
USE CODE: SPRING20
Get 20% OFF Your First Order!
Home
Search
Cart
Settings


spring season
Analytics | DebugView
Debug Device 1
Xiaomi M2003J15SC
3:38 PM 15
3:34 PM 13
3:30 PM 11
3:26 PM 4
3:21 PM 4
3:17 PM 3
3:15 PM 4
3:08 PM 13
0 new
3:38:26 PM screen_view
screen_view
view_promotion
installation_id_debug
view_cart
app_opened
3:38:25 PM screen_view
3:38:24 PM login
9s
3:38:15 PM screen_view
3:38:14 PM
1s
3:38:13 PM
view_promotion
Parameters
User properties
Items
first_open_time 1
non_personalized_ads 1
installation_id_debug 1
f56WJJZ0Rkq2rvXp5DICln
TOP EVENTS
45 TOTAL
LAST 30 MINS
20
0
25
screen_view 25
view_promotion 6
app_open 4
view_cart 3
app_opened 3
login 3
select_promotion 1
USER PROPERTIES ACTIVE NOW
first_open_time 7:30 PM
non_personalized_ads 0
user_id 123456
installation_id_debug f56WJJZ0...p5DICln


Debug Device
1
Xiaomi M2003J15SC
0
3:42 PM
3:39 PM
3:38 PM
15
3:36 PM
3:35 PM
0
new
3:38:26 PM
screen_view
view_promotion
installation_id_d
view_cart
app_opened
screen_view
view_promotion
Parameters
User properties
Items
first_open_time
1
non_personalized_ads
1
installation_id_debug
1
f56WJJZ0Rkq2rvXp5DICln